Multi-Factor Stock Selection System

System Architecture & API Reference

Flask + SQLAlchemy + Redis + WebSocket

A-Share Quantitative Analysis Platform

1. Project Overview

The Multi-Factor Stock Selection System is a comprehensive quantitative analysis platform for China A-share markets. It integrates factor computation (18+ built-in factors), machine learning modeling (Random Forest, XGBoost, LightGBM), automated factor optimization via Rank IC analysis, portfolio optimization (Mean-Variance, Risk Parity), backtesting, real-time WebSocket data streaming, and natural language Text2SQL queries. The system is built with Python 3.13 and Flask 3.1, using MySQL for persistence and Redis for caching.

- Factor Management: 18 built-in factors (momentum, volatility, PE/PB, ROE, money flow, etc.)
- ML Modeling: RF/XGBoost/LightGBM for stock return prediction with automated feature importance
- Factor Optimization: Rank IC-based automatic factor selection and weight calculation
- Stock Scoring: Multi-method scoring (equal/IC-weighted/ML-ensemble/manu al-weight)
- Portfolio Optimization: CVXPY/Scipy solvers for mean-variance, risk parity, equal weight
- Backtesting: Full strategy engine with performance metrics, benchmark comparison
- Real-time Analysis: Flask-SocketIO + eventlet for minute-level indicator streaming
- Text2SQL: jieba NLP + LLM for natural language to SQL query conversion
- Data Sources: Tushare (API), Baostock (free), akshare (HTTP scraping)

2. System Architecture

The system follows a classic three-layer architecture with Flask application factory pattern. 13 blueprints register API and page routes. Services use lazy-initialized singleton pattern. Data layer uses SQLAlchemy ORM with 19+ models mapping to MySQL tables.

2.1 Application Factory

```
# app/__init__.py
def create_app(config_name='default'):
    app = Flask(__name__)
    app.config.from_object(config[config_name])
    db.init_app(app)
    socketio.init_app(app, message_queue=...)
    CORS(app)

    # Register 13+ blueprints
    from app.api import api_bp              # /api/*
    from app.api.ml_factor_api import ml_factor_bp  # /api/ml-factor/*
    from app.api.text2sql_api import text2sql_bp
    from app.api.realtime_analysis import realtime_analysis_bp
    from app.routes.ml_factor_routes import ml_factor_routes  # /ml-factor/*
    app.register_blueprint(api_bp, url_prefix='/api')
    app.register_blueprint(ml_factor_bp)
    app.register_blueprint(ml_factor_routes)
    ...
    return app
```

2.2 Three-Layer Design

- API Layer (app/api/): 11 blueprint modules handling HTTP requests/responses with JSON
- Service Layer (app/services/): 17 service classes containing core business logic
- Data Layer (app/models/): 19 SQLAlchemy ORM models mapping to MySQL tables
- Frontend (app/templates/ + app/static/): Jinja2 + Bootstrap 5 + ECharts + Axios

2.3 Technology Stack

- Web: Flask 3.1 + Flask-SocketIO 5.6 + Flask-RESTful + Flask-CORS
- Database: MySQL + PyMySQL + SQLAlchemy 2.0 ORM + Redis 7
- ML: scikit-learn 1.8 + XGBoost 3.2 + LightGBM 4.6
- Optimization: CVXPY 1.8 + SciPy 1.17 + clarabel solver
- Data: pandas 3.0 + numpy 2.4 + scipy + statsmodels
- NLP: jieba 0.42 (Chinese word segmentation)
- Realtime: APScheduler + eventlet + Flask-SocketIO
- Frontend: Jinja2 + Bootstrap 5.1 + ECharts 5.4 + Axios 1.5

3. Database Schema

The database (stock_cursor) contains 20+ tables organized into categories:

3.1 Core Data Tables

```
stock_basic               # 5,473 A-share stocks (code, name, industry, area, list_date)
stock_daily_history       # 1,472,328 rows of daily OHLCV (2025-01 ~ 2026-02)
stock_daily_basic         # 376,125 rows of daily fundamentals (PE, PB, turnover, MV)
stock_factor              # 1,130,126 rows of technical indicators (MACD, KDJ, RSI, etc.)
stock_moneyflow           # 459,389 rows of money flow by order size
stock_cyq_perf            # 218,073 rows of chip distribution data
stock_income_statement    # Financial income statements (TTM)
stock_balance_sheet       # Financial balance sheets
stock_trade_calendar      # 4,383 trading calendar dates (2024-2026)
```

3.2 Factor & ML Tables

```
factor_definition       # Custom factor definitions (id, formula, type, params)
factor_values           # 8,372,406 rows of computed factor values (ts_code + date + factor_id)
factor_effectiveness    # IC analysis results (factor_id + eval_date + forward_period PK)
ml_model_definition     # ML model metadata (type, factor_list, params, feature_importance)
ml_predictions          # 23,846 rows of model predictions (ts_code + date + model_id PK)
```

3.3 Real-time & Text2SQL Tables

```
realtime_indicator      # Real-time technical indicators (minute-level)
trading_signal          # Trading signal records with confidence scores
risk_alert              # Risk alerts with severity levels
portfolio_position      # Portfolio holdings and weight tracking
realtime_report         # Analysis reports generated in real-time
table_metadata          # Text2SQL table schema metadata
field_metadata          # Text2SQL field descriptions
query_template          # Text2SQL query templates
query_history           # Text2SQL query execution history
business_dictionary     # Chinese-English business term mappings
```

4. Factor Engine

The FactorEngine (app/services/factor_engine.py, 900 lines) computes 18 built-in factors across four categories: technical, fundamental, money flow, and chip distribution. Custom factors can be registered via the API. Factor values are stored with Z-scores and percentile rankings for cross-sectional comparison.

4.1 Built-in Factors

- Technical (6): momentum_1d/5d/20d, volatility_20d, volume_ratio_20d, price_to_ma20
- Fundamental (7): pe_percentile, pb_percentile, ps_percentile, roe_ttm, roa_ttm, revenue_growth, profit_growth
- Money Flow (3): money_flow_strength, big_order_ratio, money_flow_momentum
- Chip Distribution (2): chip_concentration, winner_rate_change

4.2 Factor Calculation Pipeline

```
# Optimized batch computation (scripts/calculate_factors.py)
# Pre-load all 1.47M stock daily records into pandas DataFrame
price_df = pd.read_sql('SELECT ts_code, trade_date, close, pre_close, vol FROM stock_daily_history',

for trade_date in all_dates:
    # Single fetch, compute all 6 technical factors in one pass
    hist = price_df[price_df['trade_date'] <= trade_dt]
    records.extend(compute_momentum_1d(day_data))
    records.extend(compute_momentum_5d(close_series))
    records.extend(compute_momentum_20d(close_series))
    records.extend(compute_volatility_20d(close_series))
    records.extend(compute_volume_ratio(vol_data, day_data))
    records.extend(compute_price_to_ma20(close_series, day_data))

    # Batch insert every 10 dates
    if i % 10 == 9:
        cursor.executemany(sql, batch); conn.commit()
```

5. Machine Learning Models

MLModelManager supports Random Forest, XGBoost, and LightGBM. Models are trained on factor values with future N-day returns as targets. After training, feature importances are persisted to the database for ensemble scoring. Model files are saved to disk (models/*.pkl).

```
# Model configuration
model_configs = {
    'random_forest': RandomForestRegressor(n_estimators=100, max_depth=10),
    'xgboost':       XGBRegressor(n_estimators=100, max_depth=6, learning_rate=0.1),
    'lightgbm':      LGBMRegressor(n_estimators=100, max_depth=6, learning_rate=0.1)
}

# Training pipeline
X, y = prepare_training_data(model_id, start_date, end_date)
X_processed, features = feature_engineering(X, y, config)  # RobustScaler + SelectKBest
X_train, X_test, y_train, y_test = train_test_split(X_processed, y, shuffle=False)
model.fit(X_train, y_train)

# Persist feature importance for ensemble scoring
model_def.feature_importance = dict(zip(features, model.feature_importances_))
db.session.commit()
joblib.dump(model, f'models/{model_id}.pkl')
```

6. Factor Optimization Engine

FactorOptimizer (app/services/factor_optimizer.py, 500+ lines) implements automatic factor selection and weight optimization based on Rank IC analysis - the standard quantitative method for evaluating factor effectiveness.

6.1 Rank IC Computation

```
# Rank IC = Spearman correlation between factor values and forward N-day returns
def compute_rank_ic(self, factor_id, trade_date, forward_period=5):
    factor_df = get_factor_exposure(factor_id, trade_date)  # N stocks x factor values
    prices = get_future_prices(trade_date, forward_period)   # N stocks x future close
    merged = factor_df.merge(prices, on='ts_code')
    merged['forward_return'] = (merged['future_close'] - merged['current_close']) / merged['current_
    ic, p_value = spearmanr(merged['factor_value'], merged['forward_return'])
    return {'ic_value': float(ic), 'sample_count': len(merged), 'success': True}
```

6.2 Rolling IC Statistics

For each factor, IC is computed on every trading date in the evaluation window. Four metrics are aggregated: IC_mean (average predictive power), IC_std (stability), IC_IR = IC_mean/IC_std (comprehensive score), and IC_win_rate (percentage of positive IC days).

6.3 Factor Selection (Two-Stage)

```
# Stage 1: IC_IR threshold filtering
valid = [f for f in data if abs(f['ic_ir']) > threshold]  # default threshold = 0.3

# Stage 2: Correlation redundancy removal (greedy algorithm)
corr_matrix = factor_values.pivot().corr(method='spearman')
for f in sorted(valid, key=lambda x: abs(x['ic_ir']), reverse=True):
    if max(|corr| with any already_selected) > max_correlation:  # default 0.7
        continue  # redundant, skip
    selected.append(f)
```

6.4 Weight Calculation

```
# Default: ic_ir_weighted - weight proportional to |IC_IR|
ic_irs = [abs(f['ic_ir']) for f in selected_factors]
weights = ic_irs / sum(ic_irs)  # normalized to sum = 1.0

# Also available: ic_mean_weighted, equal_weight

# Example output:
# {'volatility_20d': 0.423, 'momentum_5d': 0.301,
#  'momentum_20d': 0.157, 'momentum_1d': 0.120}
```

6.5 API Endpoints

```
POST /api/ml-factor/factors/auto-weight
# 8 parameters, all optional with defaults:
#   evaluation_date, start_date, forward_period, weight_method
#   ic_ir_threshold, max_correlation, min_factors, max_factors

GET  /api/ml-factor/factors/effectiveness?factor_id=&forward_period=&limit=

POST /api/ml-factor/factors/optimize
# Full pipeline: IC analysis + selection + weights
```

7. Stock Scoring Engine

StockScoringEngine supports four scoring methods. rank_ic and ml_ensemble were previously TODO stubs and are now fully implemented with automatic weight fetching.

```
scoring_methods = {
    'equal_weight':  lambda scores, w: scores.mean(axis=1),
    'factor_weight': lambda scores, w: (scores * normalized(w)).sum(axis=1),
    'rank_ic':       calls FactorOptimizer -> gets IC-optimized weights -> factor_weight,
    'ml_ensemble':   reads model_def.feature_importance -> averages across models -> factor_weight
}
```

7.1 Data Flow

```
FactorValues (DB)
  -> calculate_factor_scores(trade_date, factor_list)
  -> pivot table: rows=ts_code, cols=factor_id, values=z_score
  -> calculate_composite_score(factor_scores, weights, method)
  -> composite_score per stock
  -> rank_stocks(composite_scores, top_n, filters)
  -> top N stocks with basic info (name, industry, area)
```

8. Backtest Engine

BacktestEngine supports factor-based and ML-based stock selection with configurable rebalancing frequency (daily/weekly/monthly/quarterly), transaction cost modeling, and comprehensive performance metrics.

```
# Backtest configuration
POST /api/ml-factor/backtest/run
{
    "strategy_config": {
        "selection_method": "factor_based",   # or "ml_based"
        "scoring_method": "rank_ic",           # equal_weight, factor_weight, rank_ic
        "factor_list": [...], "top_n": 20,
        "optimization": {"method": "equal_weight"},
        "transaction_cost": 0.001
    },
    "start_date": "2025-01-03", "end_date": "2026-02-12",
    "initial_capital": 1000000, "rebalance_frequency": "monthly"
}

# Performance metrics returned:
# total_return, annualized_return, sharpe_ratio, max_drawdown
# calmar_ratio, volatility, win_rate
# portfolio_values, daily_returns, daily_positions, daily_turnover
```

8.1 Scoring Method Integration

When scoring_method="rank_ic", the backtest engine calls _rank_ic_scoring on each rebalance date. This automatically fetches IC-optimized weights from FactorOptimizer. When the scoring_method is omitted or "equal_weight", all factors are weighted equally.

9. Portfolio Optimization

PortfolioOptimizer uses CVXPY convex optimization and SciPy numerical optimization to solve for optimal portfolio weights given expected returns and risk constraints.

- Mean-Variance: maximize(expected_return - 0.5 * risk_aversion * variance) via CVXPY ECOS solver

- Risk Parity: minimize squared difference from equal risk contribution via scipy.optimize
- Equal Weight: simple 1/N allocation
- Risk model: Ledoit-Wolf shrinkage estimator for covariance matrix estimation
- Factor Neutral and Black-Litterman: TODO stubs, fallback to mean-variance

10. Real-time Analysis System

The real-time module uses Flask-SocketIO + eventlet for WebSocket streaming of minute-level market data, technical indicators, trading signals, and risk alerts.

- Data: stock_minute_data table (5/15/30/60 min K-lines) via Tushare stk_mins API
- Indicator Engine: computes 20+ real-time technical indicators
- Signal Engine: generates buy/sell signals based on indicator crossovers
- Risk Manager: monitors position risk, stop-loss, and VaR
- Monitor Dashboard: WebSocket push of real-time metrics to browser
- Report Generator: automated daily/weekly analysis reports

11. Text2SQL Natural Language Query

The Text2SQL module converts Chinese natural language queries into SQL using jieba word segmentation and LLM-powered schema understanding.

```
# Query pipeline
User query: "show me stocks with PE < 15 and ROE > 10%"
  -> NLP Processor (jieba tokenization + entity extraction)
  -> SQL Generator (LLM + table metadata + query templates)
  -> "SELECT * FROM stock_daily_basic WHERE pe_ttm < 15 AND roe_ttm > 10"
  -> Execute against MySQL -> Return results

# Components:
# nlp_processor.py: Chinese word segmentation, entity recognition
# llm_service.py: OpenAI-compatible API integration (Aliyun DashScope qwen-coder by default)
# sql_generator.py: Template-based SQL generation with LLM refinement
# text2sql_engine.py: Orchestration of the full pipeline
```

12. Data Sources & Pipeline

12.1 Available Sources

- Tushare (tushare.pro): API token required, supports stock_basic, daily, stk_mins, moneyflow
- Baostock (baostock.com): Free, no token, Python 3.13 compatible
- akshare (HTTP): Wraps public web sources (Sina, EastMoney), IP rate-limited
- SQL Dumps: Pre-loaded MySQL dumps covering 2025-01 ~ 2026-02 (8 tables, 300MB+)

12.2 Data Pipeline

```
# Data import flow
SQL dumps (stock_data/*.sql)
  -> mysql < file.sql (DROP TABLE + CREATE + INSERT)
  -> Python fast_import.py (batch INSERT 10k rows/transaction, 20x faster)

# Factor calculation flow
stock_daily_history (1.47M rows)
  -> scripts/calculate_factors.py
  -> 6 technical factors x 272 trading days
  -> factor_values (8.37M rows)

# IC analysis flow
factor_values + stock_daily_history
  -> FactorOptimizer.analyze_all_factors()
  -> factor_effectiveness (IC stats per factor per period)
  -> auto-weight API endpoint
```

13. Frontend Pages

- /ml-factor: Factor management dashboard (list, create, calculate custom factors)

- /ml-factor/scoring: Stock scoring (factor-based, ML-based, portfolio integrated)
- /ml-factor/models: ML model management (create, train, predict, evaluate)
- /ml-factor/portfolio: Portfolio optimization and rebalancing
- /ml-factor/analysis: Factor contribution and sector analysis reports
- /ml-factor/backtest: Backtest configuration with scoring method selector, auto-weight preview, and results visualization
- /ml-factor/optimization: Factor optimization workflow (IC analysis -> weights -> backtest comparison)
- Real-time analysis pages: monitor dashboard, indicators, signals, risk, reports

All pages use Jinja2 template inheritance from base.html, Bootstrap 5.1 with custom financial theme CSS, ECharts for interactive charts, and Axios for AJAX API calls. The layout follows container > row > column with metric cards, data tables, and modals.

14. Complete API Index

```
GET  /api/ml-factor/factors/list             # List all factors
POST /api/ml-factor/factors/calculate          # Calculate factor values
POST /api/ml-factor/factors/custom             # Create custom factor
POST /api/ml-factor/factors/optimize           # Full factor optimization   [NEW]
GET  /api/ml-factor/factors/effectiveness      # IC effectiveness report    [NEW]
POST /api/ml-factor/factors/auto-weight        # Auto-compute optimal weights [NEW]
POST /api/ml-factor/models/create              # Create ML model
POST /api/ml-factor/models/train               # Train model
POST /api/ml-factor/models/predict             # Predict with model
POST /api/ml-factor/models/evaluate            # Evaluate model
GET  /api/ml-factor/models/list                # List all models
POST /api/ml-factor/scoring/factor-based       # Factor-based stock scoring
POST /api/ml-factor/scoring/ml-based           # ML-based stock scoring
POST /api/ml-factor/analysis/factor-contribution # Factor contribution analysis
POST /api/ml-factor/analysis/sector            # Sector analysis
POST /api/ml-factor/batch/calculate-and-score  # Batch: calculate + score
POST /api/ml-factor/batch/train-and-predict    # Batch: train + predict
POST /api/ml-factor/portfolio/optimize         # Portfolio optimization
POST /api/ml-factor/portfolio/rebalance        # Portfolio rebalancing
POST /api/ml-factor/portfolio/integrated-selection # Integrated selection + optimization
POST /api/ml-factor/backtest/run               # Run backtest
POST /api/ml-factor/backtest/compare           # Compare multiple strategies
```